Asset Pulse — Docker Desktop deployment guide

This guide walks a non-developer through running Asset Pulse on a single laptop using **Docker Desktop**. No manual Python, Node.js, or PostgreSQL installation is required — Docker provisions everything.

The stack consists of three containers managed by **Docker Compose**:

Service	Image / build	Internal address	Host port
`db`	`postgres:17-alpine`	`db:5432`	(none, optional `5433`)
`backend`	`./backend/Dockerfile`	`backend:8000`	`8000` (debug)
`frontend`	`./frontend/Dockerfile`	`frontend:80` (Nginx)	`8080`

The frontend container also reverse-proxies `/api/*` to the backend, so the **only URL you need is** `http://localhost:8080`.

---

1. Install / open Docker Desktop

1. Download Docker Desktop:

• macOS / Windows: <https://www.docker.com/products/docker-desktop/>

• Linux: <https://docs.docker.com/desktop/install/linux/>

2. Run the installer and accept the defaults.
3. Launch Docker Desktop and wait until the whale icon in the tray is steady

(not animating). On Windows, accept the WSL 2 prompt if it appears.

4. Verify in a terminal:

```
docker --version
docker compose version
```

You should see versions for both. If `docker compose version` errors out, restart Docker Desktop.

---

2. Extract the project ZIP

1. Unzip `asset_pulse_docker_desktop.zip` to a folder of your choice.

• Recommended path: `~/projects/asset-pulse` (macOS / Linux) or

`C:\projects\asset-pulse` (Windows). Avoid paths with spaces, OneDrive, or iCloud Drive — those can cause file-permission and lock issues with Docker bind mounts.

2. Open a terminal **in the extracted folder**:

```
cd ~/projects/asset-pulse        # macOS / Linux
cd C:\projects\asset-pulse        # Windows (PowerShell or cmd)
```

3. Confirm you're in the right place — `ls` (or `dir` on Windows) should show

`docker-compose.yml`, `backend/`, `frontend/`, `db/`.

---

3. Create your `.env` file

Compose reads database credentials from a `.env` file in the repo root.

```
# macOS / Linux
cp .env.docker.example .env

# Windows (cmd)
copy .env.docker.example .env
```

Open `.env` in any text editor and **change** `POSTGRES_PASSWORD` from `changeme` to a strong password. The other values are fine as-is.

> The `.env` file is excluded from git, so your password is never committed.

---

4. Build and start the stack

From the repo root:

```
docker compose up --build
```

The first run will:

1. Pull `postgres:17-alpine` and `node:20-alpine` / `nginx:1.27-alpine` base images.
2. Build the backend image (installs FastAPI, SQLAlchemy, psycopg2).
3. Build the frontend image (runs `npm ci && npm run build`, then copies the

static bundle into Nginx).

4. Initialize the database from `db/001_init.sql` (only happens on a fresh

volume — see the *reset* section below).

5. Start all three containers.

Expect 3–5 minutes the first time. Subsequent runs reuse cached layers and start in seconds.

When you see lines like:

```
assetpulse_db        | LOG:  database system is ready to accept connections
assetpulse_backend   | INFO:     Uvicorn running on http://0.0.0.0:8000
assetpulse_frontend  | ... start worker process
```

you're up.

To run in the background instead, use `docker compose up --build -d`.

---

5. Open the application

Visit:

```
http://localhost:8080
```

That's the entire app — the Nginx container serves the React UI and proxies API calls to the backend automatically. There is no separate frontend port to remember.

---

6. Health checks

- **Through the frontend (always works once stack is up):**

```
http://localhost:8080/api/health
```

- **Direct to backend (only if port 8000 is mapped — it is, in the default `docker-compose.yml`):**

```
http://localhost:8000/api/health
```

A healthy response looks like:

```
{ "status": "ok", "database": "postgres", "database_url_redacted": "postgresql://db:5432/assetpulse" }
```

If `database` says `sqlite-fallback`, the backend couldn't reach Postgres — see *Troubleshooting* below.

---

7. Stop, start, rebuild

Goal	Command
Stop everything (keep data)	`docker compose down`
Stop without removing containers	`docker compose stop`
Start an existing stack	`docker compose start`
Rebuild after code changes	`docker compose up --build`
Rebuild a single service	`docker compose build backend`
Restart one service	`docker compose restart backend`
Tear down and **delete the database**	`docker compose down -v`

`docker compose down` keeps the named volume `assetpulse_db_data`, so your database survives. `down -v` deletes that volume as well — only use it when you want a fresh DB.

---

8. View logs

```
# Tail logs for everything
docker compose logs -f

# Just one service
docker compose logs -f backend
```

```
docker compose logs -f frontend
docker compose logs -f db

# Last 200 lines, no follow
docker compose logs --tail=200 backend
```

`Ctrl+C` exits the log follower; the containers keep running.

---

9. Reset the database volume

The Postgres init SQL (`db/001_init.sql`) only runs the **first time** the volume is created. To start over with empty tables:

```
docker compose down -v
docker compose up --build
```

This deletes `assetpulse_db_data`, recreates it, replays `001_init.sql`, and the backend re-seeds sample data on first boot.

---

10. Backup and restore the database

The DB lives in the named volume `assetpulse_db_data` and is reachable from within the `db` container. The two recipes below run `pg_dump` / `pg_restore` **inside the running container**, so you don't need PostgreSQL installed on your host.

> All commands assume the stack is running (`docker compose up -d`).

Backup (plain SQL — easy to inspect)

```
# macOS / Linux
docker compose exec -T db \
    pg_dump -U assetpulse_user -d assetpulse > backup.sql

# Windows (PowerShell)
docker compose exec -T db `
    pg_dump -U assetpulse_user -d assetpulse > backup.sql
```

Backup (custom format — smaller, faster restore)

```
docker compose exec -T db \
    pg_dump -U assetpulse_user -d assetpulse -Fc > backup.dump
```

Restore from `backup.sql`

```
# Wipe and recreate the database first to avoid duplicate-row errors:
docker compose exec -T db \
    psql -U assetpulse_user -d postgres \
    -c "DROP DATABASE IF EXISTS assetpulse;" \
    -c "CREATE DATABASE assetpulse OWNER assetpulse_user;"

# Then load the dump:
docker compose exec -T db \
    psql -U assetpulse_user -d assetpulse < backup.sql
```

Restore from `backup.dump`

```
docker compose exec -T db \
    pg_restore -U assetpulse_user -d assetpulse --clean --if-exists < backup.dump
```

After restoring, `docker compose restart backend` to clear any stale SQLAlchemy connections.

---

11. Troubleshooting

"Port 8080 is already in use"

Something else on your laptop is using port 8080 (often a local web server or another Docker stack).

Either stop the offending process, or change the host port in `docker-compose.yml`:

```
  frontend:
    ports:
      - "9090:80"   # then use http://localhost:9090
```

The same applies if `8000` (backend debug) clashes.

"Cannot connect to the Docker daemon" / "is the docker daemon running?"

Docker Desktop is not started. Open the Docker Desktop app and wait for the whale icon to stop animating, then re-run `docker compose up`.

"Database is uninitialized and superuser password is not specified"

Your `.env` file is missing or empty. Re-create it:

```
cp .env.docker.example .env
```

and re-run `docker compose up --build`.

"FATAL: role 'assetpulse_user' does not exist" after editing `.env`

PostgreSQL only honors `POSTGRES_USER` / `POSTGRES_PASSWORD` when it **creates** the data directory. If you change those values after the volume has been initialized, the old credentials persist.

Fix by resetting the volume:

```
docker compose down -v
docker compose up --build
```

(this deletes existing DB data — back up first if you care).

Backend health check shows `sqlite-fallback`

The backend couldn't reach Postgres. Common causes:

- `db` container is still starting → wait 10–20 seconds and refresh.
- `POSTGRES_PASSWORD` differs between `.env` and the existing volume → see

the previous item.

- Inspect logs: `docker compose logs db backend`.

Frontend returns 502 Bad Gateway on `/api/...`

Nginx couldn't reach the backend service.

```
docker compose ps          # is `backend` Up (healthy)?
docker compose logs backend
```

If the backend is restarting, fix its error first; the frontend will start working as soon as the backend is healthy again.

Windows path / line-ending issues

- Avoid storing the project under OneDrive or paths containing accents /

non-ASCII characters — Docker bind mounts can fail silently.

- If `db/001_init.sql` errors with `unexpected character`, your editor may

have rewritten it with CRLF + BOM. Re-extract the ZIP or run `git config --global core.autocrlf input` before cloning.

- Make sure WSL 2 integration is enabled in Docker Desktop

(*Settings* → *Resources* → *WSL Integration*) — required on Windows 10/11.

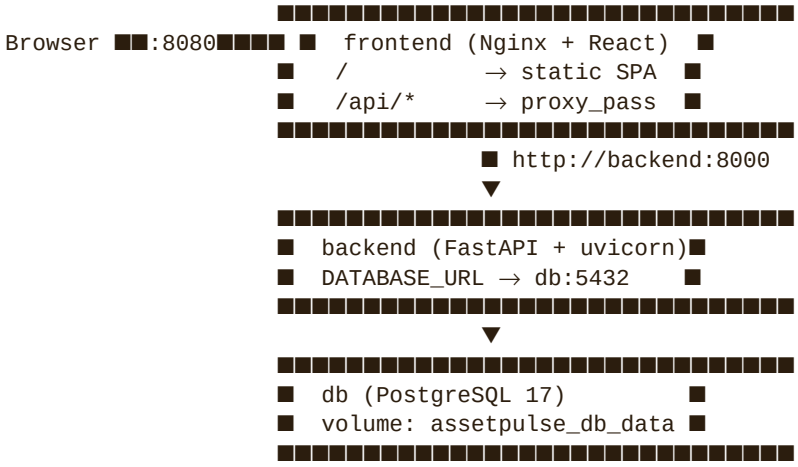
"I changed code but the running app doesn't update"

The compose file builds images at startup; it doesn't hot-reload. After editing source code:

```
docker compose up --build
```

For an active dev loop, run the frontend with `npm run dev` (port 5173) against the dockerized backend instead — `CORS_ORIGINS` in `.env` already allows `http://localhost:5173`.

Architecture summary



That's it — `docker compose up --build` and you're running the full stack.